Ochrona danych w systemach informatycznych

Laboratorium 4
Funkcje skrótu

Jakub Ciszewski

Semestr 2026Z

Plan i zasady

90 minut (2 × 45). Trzy zadania po 1 pkt.

Dowolny język. Cały kod piszesz samodzielnie.
Sumy: 20 min. Hasła: 25 min. Kolizje: 25 min.

ISOD, dokumentacja i własne notatki są dozwolone.
Podczas zajęć wyłącz czaty i podpowiedzi LLM.

Kod na GitHubie. Jeden raport TXT/PDF/MD.
Raport i porządki w repozytorium także po zajęciach.

Funkcja skrótu: trzy odporności

Własność	Trudne dla atakującego
Przeciwobraz	dla y znaleźć x: h(x) = y
Drugi przeciwobraz	dla x znaleźć x′ ≠ x: h(x′) = h(x)
Kolizja	znaleźć dowolne x ≠ x′: h(x) = h(x′)

Wynik ma stałą długość, niezależną od długości danych.

Algorytmy: długość i zastosowanie

Algorytm	Skrót	Do zabezpieczeń
MD5	128 bitów	Nie: praktyczne kolizje
SHA-1	160 bitów	Nie: praktyczne kolizje
SHA-256	256 bitów	Tak
SHA3-256	256 bitów	Tak

Efekt lawinowy: zmiana 1 znaku zmienia ok. połowę bitów.

Zadanie 1: sumy kontrolne

SHA256SUMS.txt: skrót hex, dwie spacje, ścieżka.

Policz SHA-256 każdego pliku.
Możesz wczytać cały plik naraz.

Dla każdego wiersza: OK, BŁĄD albo BRAK.
Brak pliku nie przerywa programu.

Ala ma kota. / Ola ma kota. — policz różne bity.

Obserwacja: suma, HMAC, podpis

Mechanizm	Błąd przypadkowy	Celowa podmiana
Suma SHA-256 obok pliku	Wykrywa	Nie: podmienia się też sumę
HMAC z tajnym kluczem	Wykrywa	Wykrywa, gdy klucz jest tajny
Podpis cyfrowy	Wykrywa	Wykrywa; sprawdzenie kluczem publicznym

Sama suma chroni, gdy pochodzi z zaufanego źródła.

Przechowywanie haseł

Jawne hasła: wyciek bazy ujawnia wszystkie konta.

Sam SHA-256(hasło): szybki, a równe hasła
dają równe skróty.

Sól: losowa, jawna, inna dla każdego konta.
Iteracje: każda próba hasła kosztuje więcej.

```
login:iteracje:sól_hex:skrót_hex
```

Dodatkowo: koszt przeszukania haseł

Alfabet, długość	Kandydatów	Entropia
a–z, 6	$3{,}1 \cdot 10^{8}$	28,2 bitu
a–z, 8	$2{,}1 \cdot 10^{11}$	37,6 bitu
a–z, A–Z, 0–9, 8	$2{,}2 \cdot 10^{14}$	47,6 bitu
a–z, A–Z, 0–9, 12	$3{,}2 \cdot 10^{21}$	71,5 bitu

Czas ≈ liczba kandydatów × czas jednej próby.

Zadanie 2: weryfikacja haseł

Wczytaj hasla.txt. PBKDF2-HMAC-SHA256 z parametrami rekordu; porównanie w stałym czasie.

Sprawdź cztery przypadki z poleceń B.

Dodaj dwa konta z tym samym hasłem i nową solą.
Porównaj sole i skróty, zweryfikuj oba konta.

W raporcie: wyniki testów i porównań.
Pomiary czasu są dodatkowe, poza punktacją.

Paradoks urodzinowy

Dopasowanie do ustalonego skrótu n-bitowego:
aż około 2^n prób.

Dowolna para o równym skrócie (kolizja):
około 2^(n/2) prób.

h24: $2^{24} \approx 16{,}8$ mln wartości, a $2^{12} = 4096$.

Dwa zbiory po k wariantów: około k · k / 2^n par.

Zadanie 3: atak białych znaków

h24 = pierwsze 3 bajty SHA-256 (6 znaków hex).

Wariant v: spacja przed \n wiersza i,
gdy (v >> i) & 1 = 1, dla i = 0…12.

Słownik: dla każdego h24 jeden numer wariantu.
Znajdź jedną parę o równym h24.

Sprawdź też wszystkie 8192 zmienione warianty:
ile pasuje do h24 niezmienionego oryginału?

Oddanie: tylko jeden plik raportu

TXT, PDF albo MD — do jednej strony.

Imię, nazwisko, indeks i język.
Link do GitHuba oraz identyfikator commitu.

Wyniki zadań i krótkie odpowiedzi z B.
Raport i porządki w repozytorium także po zajęciach.

Na GitHubie: cały kod i README z uruchomieniem.
Repozytorium musi być dostępne dla prowadzącego.

Pytania przy odbiorze

Dlaczego suma obok pliku nie chroni przed podmianą?

Co daje sól, a co liczba iteracji?

Dlaczego kolizję znaleźć łatwiej niż przeciwobraz?

Za zadanie: 0,50 pkt kod, 0,25 wyniki,
0,25 poprawne wyjaśnienie.

Materiały do przygotowania

ISOD: Laboratorium 4 (Funkcje hash).
Wykład: Jednokierunkowe funkcje skrótu.

Dokumentacja wybranej biblioteki: SHA-256,
PBKDF2, porównanie w stałym czasie, losowanie.

NIST FIPS 180-4: SHA-256
OWASP: Password Storage Cheat Sheet

Przed zajęciami: środowisko i GitHub.
Szczegółowa lista w A-przygotowanie.txt.